University Student Service & Queue System

TERMINAL ASSESSMENT 2 /
MILESTONE 2 STUDENT
SECTION: 2.4

Members:

Bernardo, Marvin
Cajayon, Mark Erick
Bulatao, Harvey
Amores, Renedel

Introduction

- A menu-driven C++ application for university service requests.
- Manages student records such as student ID, name, service, and priority.
- Organizes regular and priority requests using queues.
- Demonstrates data structures, searching, sorting, and dynamic memory management.

Project Problem and Objectives

Project Problem:

- Manual handling of student records and requests can be difficult to organize.
- Staff need a way to find and update records efficiently.
- Requests must be processed in a clear order, including priority cases.

Objectives:

- Build a C++ system that manages student records.
- Process regular and priority requests.
- Demonstrate algorithm performance.

System Requirements

- Student Records: Add, display, search, update, and delete.
- Request Processing: Regular queue and priority queue.
- Undo/Audit: Record recent ADD, UPDATE, and DELETE actions.
- Algorithms: Linear Search, Binary Search, Bubble Sort, Selection Sort, and Insertion Sort.
- Validation: Check inputs and prevent duplicate student IDs.
- Memory Management: Release dynamically allocated nodes and arrays when no longer needed.

Data Structure Design

Doubly Linked List

Stores the main student records. Each node links to the next and previous node.

Linked Stack

Stores recent actions for undo and audit history. Follows Last-In, First-Out (LIFO).

Circular Queue

Processes regular requests in First-In, First-Out (FIFO) order.

Priority Queue + Dynamic Arrays

Places priority 1 requests before priority 0 requests. Temporary arrays support searching, sorting, and benchmarking.

Linked List Implementation

Stores students' records in Doubly Linked List. Records are added, deleted, searched, updated, and displayed in O(n) time as the operation entails a check or traversal of nodes.

Stack Implementation

Saves ADD, UPDATE and DELETE actions for undo in a linked AuditStack in LIFO order. Push, pop and peek all have O(1) time complexity, and displaying has O(n) time complexity.

Queue/ Priority Queue

Regular requests have a 100 length circular queue that implements FIFO (O(1) enqueue, dequeue). Priority requests are implemented in a sorted linked list with priority 1 being higher than priority 0 ($O(n)$ enqueue, and $O(1)$ dequeue).

Search Algorithms

1. Linear Search
Checks each record one by one until the target is found.
Does not require sorted data.
Time Complexity: O(n)

2. Binary Search
Repeatedly divides the search range into two halves.
Requires sorted data by student ID.
Time Complexity: O(log n)

Purpose: To find student records efficiently and compare the performance of different searching methods.

Sorting Algorithms

1. Bubble Sort

Compares adjacent elements and swaps those in the wrong order.
Time Complexity: $O(n^2)$

2. Selection Sort

Finds the smallest element and places it in its correct position.
Time Complexity: $O(n^2)$

3. Insertion Sort

Inserts each element into its proper position in the sorted portion.
Time Complexity: $O(n^2)$

Purpose: *To organize student records and evaluate the performance of different sorting algorithms*

Benchmarking Results

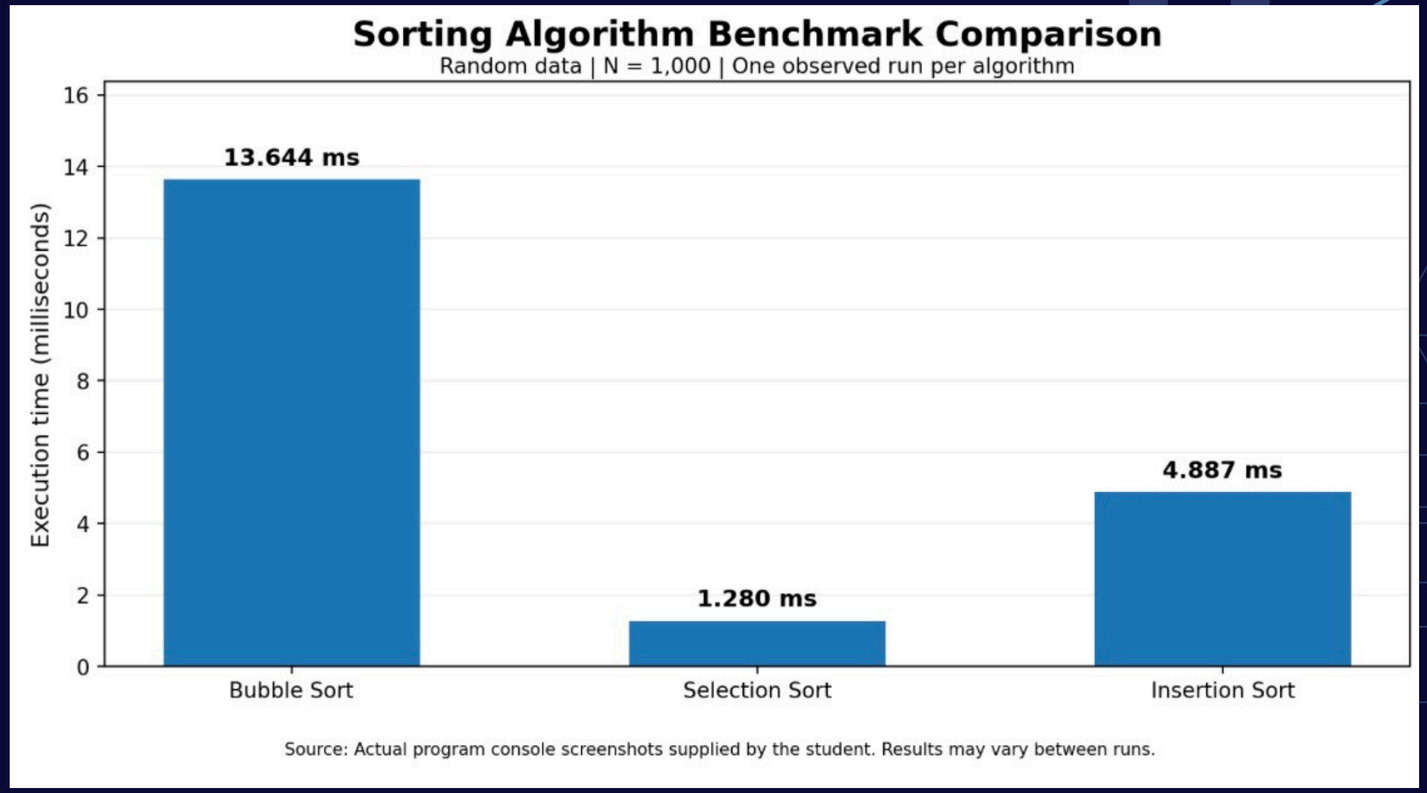
Test Configuration

Dataset size: 1,000 records

Data profile: Random

Algorithms: Bubble Sort, Selection Sort, Insertion Sort

Metrics: Comparisons, swaps/moves, and execution time



The benchmarking results show that Selection Sort was the fastest, while Insertion Sort had the fewest comparisons. These results highlight the importance of evaluating sorting algorithms through execution time and operation counts.

DYNAMIC MEMORY MANAGEMENT

Code Component	Memory Allocated?	Memory Released?	Method / Function Used	Verified (No Leak)?
Linked List Nodes	Yes – new ListNode	Yes in remove/clear	delete removed nodes; destructor clears remaining nodes	Verified
Stack memory tack memory	Yes – new stack node	Yes in pop/clear	Pop deletes removed node; destructor clears stack	Verified
Queue memory	Yes – dynamic array	Yes	Circular queue destructor uses delete[]	Verified

DYNAMIC MEMORY MANAGEMENT

Code Component	Memory Allocated?	Memory Released?	Method / Function Used	Verified (No Leak)?
Priority queue	Yes – new queue node	Yes in remove/clear	delete removed nodes; destructor clears remaining nodes	Verified
Dynamic arrays	Yes – new Student[n]	Yes in pop/clear	Pop deletes removed node; destructor clears stack	Verified
System destructors	N/A - No new memory allocated during destruction	Dynamically allocated resources array	Delete[] resources;	Verified

TESTING WORKSHEET

Test #	Target Module/Input Data	Expected Output/Actual Output
1	Add a new ID with non-empty name and service	A record is added and a success message appears.
2	Add an ID already in the list	uplicate ID is rejected
3	Display records when list has entries	All current records display in table-like console columns.

TESTING WORKSHEET

```
Enter choice: 1
Student ID (1-999999): 1
Full name: Harvey Bulatao
Service request (Assessment/Clearance/Transcript): Assessment
Priority? (1 = Yes, 0 = No): 0
Student added successfully.
```

1

2

```
Enter choice: 1
Student ID (1-999999): 1
Full name: Mark Erick Cajayon
Service request (Assessment/Clearance/Transcript): Clearance
Priority? (1 = Yes, 0 = No): 0
That student ID already exists.
```

```
Enter choice: 2
ID        Name                    Service                 Priority
-------------------------------------------------------------------
1         Harvey Bulatao          Assessment              Regular
2         Marvin Bernardo         Clearance               Regular
```

3

TESTING WORKSHEET

Test #	Target Module/Input Data	Expected Output/Actual Output
4	Search for existing ID	The student's details display
5	Search for nonexistent ID	Student not found message appears.
6	Update an existing student using a unique replacement ID	The record is updated and UPDATE action is pushed.

TESTING WORKSHEET

```
ID: 1
Name: Harvey Bulatao
Service: Assessment
Priority: Regular
```

4

5

```
Enter choice: 3
Student ID to search: 3
Student not found.
```

```
Student ID to update: 1
Student ID (1-999999): 3
Full name: Harvey Bulatao
Service request (Assessment/Clearance/Transcript): Clearance
Priority? (1 = Yes, 0 = No): 1
Student updated.
```

6

TESTING WORKSHEET

Test #	Target Module/Input Data	Expected Output/Actual Output
7	Try updating a record to another existing ID	Update is rejected.
8	Delete an existing student	The record is deleted and DELETE action is pushed.
9	Undo ADD / UPDATE / DELETE	Latest record action is reversed where possible

TESTING WORKSHEET

```
oice: 4
D to update: 3
D (1-999999): 2
e: Harvey Bulatao
equest (Assessment/Clearance/Transcript): Trans
 (1 = Yes, 0 = No): 0
iled. The replacement ID may already exist.
```

7

8

```
Enter choice: 5
Student ID to delete: 3
Student deleted.
```

```
 choice: 8
completed: DELETE for student I
```

9

TESTING WORKSHEET

Test #	Target Module/Input Data	Expected Output/Actual Output
10	Add regular student to queue	Students enter a circular queue; FIFO order is preserved.
11	Add priority student to queue	Student enters priority queue; priority 1 is first
12	Process empty queue	Queue is empty message appears

TESTING WORKSHEET

10

```
2. Process regular queue
3. Process priority queue
4. Display regular queue
5. Display priority queue
Choose: 2
Now processing:
ID: 2
Name: Marvin Bernardo
Service: Clearance
```

11

```
Enter choice: 6
1. Add a student to a queue
2. Process regular queue
3. Process priority queue
4. Display regular queue
5. Display priority queue
Choose: 3
Now processing:
ID: 1
Name: Harvey Bulatao
Service: Assessment
```

12

```
5. Display priority queue
Choose: 2
Queue is empty.
```

TESTING WORKSHEET

Test #	Target Module/Input Data	Expected Output/Actual Output
13	Enter invalid menu number or letters for integer prompt	Input is rejected and prompt repeats.
14	Sort with fewer than two records	The program asks for at least two records.
15	Run benchmark	Chosen profile and sorting algorithm metrics display.
16	Exit program	Program exits and destructors clean allocated data

TESTING WORKSHEET

```
5. Display priority queue
Choose: 2
Queue is empty.
```

13

```
Enter choice: a
Invalid input. Enter a number from 0 to 10.
```

14

```
Enter choice: 9
At least two student records are needed.
```

TESTING WORKSHEET

15

```
BENCHMARK RESULT
Algorithm: Insertion Sort
Dataset size: 100
Profile: Already sorted
Comparisons: 99
Swaps/moves: 0
Execution time (ms): 0.007
```

16

```
0. Exit
Enter choice: 0
Program ended. Dynamic memory will be cleaned up.
```

Bug and Debugging Log

No	Finding / limitation	Effect	Recommended action
1	Undoing a DELETE restores the student by calling add(), which appends it to the tail.	The record is restored, but its original position in the linked list is not preserved.	If exact restoration order is required, store the original position or neighbor references/IDs.
2	undoLast() pops the audit action before attempting the reversal.	If reversal fails, that action has already been removed from the stack and cannot be retried through the same entry.	Peek or validate the action before permanently removing it, or push it back when undo fails.
3	Queue entries store copies of Student data.	If a student record is later updated or deleted, an already queued copy may show the earlier details and can still be processed.	Decide whether queues should hold snapshots or references/IDs, and document that behavior.

Bug and Debugging Log

No	Finding / limitation	Effect	Recommended action
4	Benchmarking measures sorting algorithms only.	No automated timing comparison is produced for linear search or binary search.	Add a search benchmark if the specification explicitly requires it.
5	Benchmark random IDs can repeat.	The benchmark may include duplicate IDs unlike the registry, which rejects duplicate IDs.	If unique benchmark IDs are required, generate unique values or shuffle a unique sequence.
6	Insertion Sort increments the swaps counter for shifts.	The printed label says swaps/moves, which is suitable, but the count is not the number of calls to swap().	Keep the label 'swaps/moves' and explain the metric in the report.

Reflection

- What was the most challenging part of the project?
- What did we learn about data structures and algorithms?
- What improvements can we make to the system in the future?
- How did teamwork help us complete the project?

Thank You For Listening